AI Agent Skills and MCP: Building the Right AI Agent Pattern

Stephen Simon

Chapter 2: Building AI Agent Skills

Turning a Repeated Workflow into a Skill

Set up the course project and build your first Agent Skill

In this chapter you work on one project for the rest of the course: the weekly launch risk review for Orbit Labs, a fictional company launching Orbit 2.0 on November 16, 2026. This article covers the one-time project setup, then walks through converting the launch review prompt into a reusable Agent Skill called `launch-risk-review`.

What you need

- **Node.js 20 or later.** Check with `node --version`.
- **An AI coding agent that supports Agent Skills.** The videos use Claude Code. Other agents that support the `SKILL.md` format work too, but folder locations may differ.
- **The exercise files for this course,** downloaded and unzipped.

One-time project setup

The exercise files contain two project folders:

- `orbit-launch` is your workspace. You open this folder in your AI agent and make all your changes here.
- `orbit-backend` stands in for Orbit Labs' company systems: a project tracker and a beta support desk. You won't use it until Chapter 3. Don't open it in your agent.

There's also a `solutions` folder with the end state of each hands-on video.

Follow these steps once:

1. Open a terminal in the `orbit-backend` folder and run `npm install`.
2. Run `npm run check`. You should see seven tools, two resources, and the line `Open tickets older than 7 days: 6`. This confirms the Chapter 3 server works. You can close the terminal.
3. Open a terminal in the `orbit-launch` folder and run `claude` to start Claude Code there. If you use a different agent, open the `orbit-launch` folder in it.

Tip: if your agent ever asks for permission to read files in `orbit-backend`, decline. In the course story those are company systems, and in Chapter 3 you'll reach them the right way, through MCP.

What's already in the workspace

- `prompts/weekly-review-prompt.md` holds the full launch review prompt: severity rules, what to ignore, evidence rules, and the output format, about 40 lines in total. Its last line attaches this week's status update.
- `inputs/` holds snapshot documents people would normally paste into a prompt: two weekly status updates, go/no-go meeting notes, and a vendor email.
- `.claude/skills/` is empty except for a README. Your Skill goes here.

Step 1: Separate this week's request from the team's know-how

Open `prompts/weekly-review-prompt.md`. Go through it line by line and ask one question: *will this change next week?*

- The last line, `@inputs/status-update-week-40.md`, changes every week. It's context for this run, so it stays in the prompt.
- The severity rules, filters, evidence rules, and format don't change. They describe how this team performs a launch review, so they move into the Skill.

Step 2: Create the Skill folder

In `orbit-launch`, create the folder `.claude/skills/launch-risk-review`. A Skill is a folder, and the folder name should match the Skill's name. The only required file is `SKILL.md`.

Step 3: Write SKILL.md

Create `.claude/skills/launch-risk-review/SKILL.md` with the following content:

```
---
name: launch-risk-review
description: Reviews product launch information (status updates, meeting
notes, vendor emails, project data) and produces a launch risk report with
severity, evidence, and next actions. Use when asked to review a launch,
check launch readiness, prepare a go/no-go recommendation, or assess how
an update affects the launch date.
---

# Launch risk review

Use this Skill to find risks to a product launch and report them the same
way every time.

## Process

1. **Find the dates.** Identify the launch date and the date of the
information you're reviewing. If the launch date isn't given, ask for it.
2. **Collect candidate risks.** Look for anything that could delay the
launch or hurt launch quality: blocked or late work, slipping
```

```
dependencies, unowned tasks, clusters of customer problems, and missing
sign-offs.
3. **Apply the filters.** Drop anything listed under "What to ignore" in
[rubric.md](rubric.md).
4. **Assign severity.** Use the rules in [rubric.md](rubric.md). When two
rules apply, use the higher severity. Do the date math explicitly when
deciding between High and Medium.
5. **Attach evidence.** Every risk needs at least one piece of evidence: a
short quote from the source, or a task, ticket, or milestone ID. If you
can't point to evidence, it isn't a finding.
6. **Write the report.** Follow [report-template.md](report-template.md)
exactly. Keep the column names and order.

## Rules

- Don't invent dates, owners, or facts. If something important is missing,
list it under "Missing information".
- Prefer the most recent source when two sources disagree, and mention the
conflict in the evidence.
- Combine duplicates. Several tickets or notes about the same underlying
problem are one risk with several pieces of evidence.
- Keep recommended actions specific and doable this week.
```

Two parts of this file do different jobs:

- **The frontmatter** (between the `---` lines) is what the agent sees before it decides to use the Skill. The `description` lists the situations where the Skill applies, so the agent can match it to requests like a go/no-go check or a vendor update, not only weekly reviews.
- **The body** is a procedure the agent follows. It also records judgment that the original prompt left unstated, such as combining duplicate reports into one risk and preferring the most recent source when documents disagree.

Step 4: Add the rubric

Create `.claude/skills/launch-risk-review/rubric.md`:

```
# Launch risk rubric

## Severity

| Severity | Use when |
|---|---|
| **High** | The risk could delay the launch by **more than two weeks**. |
| **Medium** | The risk could delay the launch by **one to two weeks**, or
would hurt launch quality without delaying it. |
| **Low** | Anything else worth tracking. |

## Escalation rules

These raise severity. They never lower it.
```

```
- **Unowned launch-critical work:** any task needed for a launch milestone
with no owner is at least **Medium**.
- **Systemic customer issue:** three or more open tickets about the same
problem, each open more than 7 days, are one risk. **High** if any of them
are P1 or P2, otherwise **Medium**.
- **Widespread blocker:** any open P1 ticket affecting more than 3
customers is **High**, regardless of age.
- **Vendor slip:** a vendor date that moves past the milestone that
depends on it is at least **Medium**. Work out the new finish date to
decide whether it's High.

## What to ignore

- Closed tickets.
- Tickets waiting on the customer (`pending_customer`).
- Cosmetic issues and feature requests, unless more than 10 customers
reported them.
- Work already marked done.
```

Every rule here is testable. “More than two weeks of delay” gives the agent something to check. “Significant delay” would not.

Step 5: Add the report template

Create `.claude/skills/launch-risk-review/report-template.md`:

````
# Report template

Use this structure exactly.

```markdown
## Launch risk review: <product> (<date of information reviewed>)

**Overall status:** <On track | At risk | Off track>
<One sentence explaining the status.>

| Risk | Severity | Evidence | Recommended next action | Suggested owner |
|---|---|---|---|---|
| <short description> | High | <quote or IDs> | <specific action this
week> | <name or "Unassigned"> |

### Missing information
- <what's missing and why it matters, or "None">
```

Status rules:

- **Off track:** two or more High risks.
- **At risk:** exactly one High risk, or three or more Medium risks.
- **On track:** anything else.
````

```
Sort the table High, then Medium, then Low.
```

Step 6: Shrink the prompt

Replace everything in `prompts/weekly-review-prompt.md` with the only line that changes from week to week:

```
Review @inputs/status-update-week-40.md for launch risks.
```

Step 7: Load the Skill and run the review

1. Start a new agent session. Skills are discovered when a session starts, so a session that was already open won't see the new Skill.
2. Ask `What skills can you use?` and confirm that `launch-risk-review` appears with its description.
3. Send the one-line request from `prompts/weekly-review-prompt.md`: `Review @inputs/status-update-week-40.md for launch risks.`

What you should see

AI output varies in wording from run to run, so compare the findings and the structure, not the exact sentences.

- The agent indicates that it's using the `launch-risk-review` Skill.
- The report opens with a heading for Orbit 2.0 and the date of the update, followed by **Overall status: At risk**.
- The pen test booking is rated **High**: it has no owner, the vendor needs six weeks of lead time, and security sign-off is due October 30.
- The onboarding docs are rated **Medium**, because the writer is out for a week.
- The payment reports appear as Low or Medium, since the update only mentions "a couple" of them.
- The dark mode request is left out, because the rubric ignores feature requests.
- The report ends with a **Missing information** section.

These are the same findings the original 40-line prompt produces. That's the goal of this step.

Why this works

You didn't change what the workflow knows. You changed where that knowledge lives.

- **One place to maintain.** When a rule changes, you edit `rubric.md` once and every future review uses it, for everyone on the team.
- **The agent knows when to use it.** The description lets the agent pick the Skill for related requests without being told.
- **It loads in layers.** The agent always sees the name and description, reads `SKILL.md` when the Skill applies, and opens the rubric and template only when the process points to them. Keeping `SKILL.md` short keeps the agent's context small.
- **The request stays about today.** The prompt now holds only what's specific to this run.

If you get stuck

From the folder that contains `orbit-launch`, `orbit-backend`, and `solutions`, run:

```
node solutions/catch-up.mjs 02_01
```

This copies the finished Skill files and the one-line prompt into `orbit-launch`. Start a new agent session afterward.